

Условия и сигналы (condvar).

Синхронизация различных потоков может быть реализована через взаимное исключение, как это было проиллюстрировано в предыдущих примерах про читателей-писателей. Однако именно для синхронизации есть и более удобный механизм условных переменных condvar. Основная идея состоит в передаче сигналов между потоками. Один поток входит в состояние ожидания сигнала, а другой поток может послать сигнал, который активирует ожидающий поток и позволит ему продолжить свою работу. Поддержка подобного механизма возложена на переменную типа pthread_cond_t. Поскольку необходимость синхронизации наиболее часто вызвана упорядочиванием доступа к ресурсу, то этот механизм также привлекает к работе и mutex, управляющий доступом к критическим секциям кода. Ожидать сигнала может не один поток, а несколько. Тогда они образуют очередь, и сигнал разблокирует один поток из этой очереди (signal) или все потоки этой очереди (broadcast). Итак, имеем

```
pthread_cond_t      // PTHREAD_COND_INITIALIZER
pthread_cond_init(pthread_cond_t *, pthread_condattr_t *);
pthread_cond_signal(pthread_cond_t *);
pthread_cond_broadcast(pthread_cond_t *);
pthread_cond_wait(pthread_cond_t *, pthread_mutex_t *);
pthread_cond_destroy(pthread_cond_t *);
```

Для случая синхронизации читателей-писателей

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; // для защиты критической секции
pthread_cond_t  c = PTHREAD_COND_INITIALIZER;  // для сигналов
```

```
void *Reader(void *arg)
{
    // в бесконечном цикле пытаемся читать
    while(true) {
        // блокируем критическую секцию чтения/записи
        pthread_mutex_lock(&m);
        // ожидаем сигнал от писателя (и открываем критическую секцию)
        pthread_cond_wait(&c, &m);
        // тут уже можем читать так как кр. секция опять заблокирована
        .....
        // все прочитали, открываем секцию
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

```
void *Writer(void *arg)
{
    // в бесконечном цикле пытаемся писать
    while(true) {
        // блокируем крит. секцию чтения/записи
        pthread_mutex_lock(&m);
        // теперь можно безопасно писать свои данные
        .....
        // закончили писать - посылаем сигнал читателю
        pthread_cond_signal(&c);
        // разблокируем критическую секцию
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

Рассмотрим варианты развития событий.

Предположим, что читатель начал работать раньше и первым заблокировал mutex m. Теперь писатель ничего делать не может, он заблокирован на своем вызове lock. Читатель же проходит дальше и блокируется на вызове wait до прихода сигнала. При этом mutex m разблокируется, и писатель получает возможность продолжить свою работу. Он блокирует mutex и пишет свои данные в рамках своей критической секции. Потом посылает сигнал и разблокирует mutex. Читатель получает сигнал (при этом снова блокируется mutex) и спокойно читает данные, после чего снимает блокировку mutex и продолжает цикл. После чего он опять блокируется либо на своем lock, если писатель успел уже приступить к работе (выполнить свой lock), либо на wait, ожидая следующего сигнала от писателя. Таким образом, каждая запись писателя попадает к читателю.

Пусть теперь первым начинает писатель. Он блокирует mutex m, пишет данные и посылает сигнал. Но читатель может к этому моменту еще даже не начать свою работу, а если начнет, то будет заблокирован на своем вызове lock. В такой ситуации сигнал писателя пройдет без последствий, поскольку его никто не ждет, читатель еще не успел вызвать свой wait. Далее писатель разблокирует mutex, читатель выйдет из lock, захватив mutex, и блокируется на вызове wait, из которого он выйдет уже только при втором сигнале от писателя. Таким образом первый набор данных для читателя будет потерян.

Пример. Файл signal.cpp моделирует работу прогресс-индикатора для продолжительных вычислений. Запускается поток HardWork, который симулирует интенсивные вычисления и 4 раза в секунду (usleep(250000)) записывает в глобальную переменную total сколько процентов работы выполнено. Поток Progress работает по схеме читателя и, получив сигнал, читает переменную total и печатает ее на экран. Писатель HardWork посылает сигнал не на каждое изменение total, а только на те, которые кратны 10.

Файл signal2.cpp показывает модификацию предыдущей задачи на три независимых прогресс индикатора, которые показывают изменение процентов с кратностями 10, 20, 40 соответственно.

Можно поэкспериментировать с взаимной очередностью запусков потоков HardWork и Progress, чтобы увидеть последствия пропуска первого сигнала как это было описано выше.

Задача. Как модифицировать этот код, чтобы первый сигнал от HardWork всегда обрабатывался независимо от очередности запусков этих потоков? Т.е. писатель должен подождать готовности читателя.

Задача. Программа запускает три потока для выполнения некоторой работы. Потоки работают разное время. Нужно обрабатывать (печатать) результат потока сразу как только конкретный поток завершит свою работу. Напомним, что результат конкретного потока доступен по вызову pthread_join, а он блокируется до завершения потока. Поэтому нельзя вызывать pthread_join просто так, поскольку можно случайно попасть на самый продолжительный поток и ждать его результата, когда остальные потоки уже давно завершатся.

Пример из пройденного. Файлы matrvect.cpp и matrvectinit.cpp, matrvectinit.h — многопоточное умножение матрицы на вектор. Распараллеливание по строкам. Код работает, но содержит ошибку. Нужно ее найти и исправить. Также иллюстрируется еще один способ измерения времени работы потоков.

Задача. Для предыдущей задачи нужно реализовать распараллеливание по столбцам.

Задача. Для задачи умножения матрицы на вектор само умножение реализовать через процедуру параллельного скалярного произведения (строки матрицы на вектор).

Задача. Реализовать блочное распараллеливание умножения матрицы на вектор. Матрица делится на “квадратные” блоки и в векторе выделяются соответствующие участки. Далее в потоках выполняется умножение “маленьких” матричных блоков на “маленькие” векторы и результаты соответствующим образом суммируются.

Задача. Распараллелить сортировку пузырьком. Массив делится на несколько участков, каждый из которых блокируется своим мьютексом. Поток блокирует участок и делает по нему один проход методом пузырька, после чего разблокирует данный участок переходит к следующему участку. Дойдя до последнего участка, потом возвращается к первому. Т.е. каждый поток протаскивают “пузырек” вверх по массиву от его начала до его конца, и если поток один, то это будет эквивалентно обычному пузырьковому алгоритму. Чтобы потоки не мешали друг другу и вводятся участки и их блокирование через мьютекс. Пока первый поток проходит по первому участку, все остальные ждут пока участок освободится. Когда он освободится (мьютекс разблокирован), в работу включается второй поток, когда и он пройдет первый участок, в дело вступает третий и т.д. Так потоки проходят по массиву “друг за другом”, по ходу блокируя участки, на которых они работают. Количество участков и количество потоков не обязательно должно совпадать. Как раз интересно посмотреть как будет зависеть ускорение от соотношения этих количеств. (PS алгоритм не имеет практического смысла, но интересен именно как упражнение по синхронизации).